AULA 3 · ETAPA 2 · TURMA LIVE · 1H30

O TP1, lido — e a imagem no grafo

Leitura do TP1 com as datas desta turma; o tópico que carrega pixels; o pipeline do seu projeto



Prof. Dácio Moreira de Souza · segunda 26/10 · Zoom · turma GRLEDCR3C2-N2-L1

CONFERÊNCIA (10 MIN)

O que a semana passada devia ter deixado pronto

1	**Tarefa da Aula 2 na dev: 4 nós, /contagem respondendo (push até ontem)**	*git log --oneline -3*
2	**Ambiente: check-ambiente.sh termina em 'Tudo pronto!'**	*tarefa da semana 1*
3	**Repositório pb-live-<usuario> clonado em ~, branch dev**	*convite aceito por e-mail*
4	**Projeto: candidato declarado no Infnet.Online (família 1–7 ou P)**	*catálogo + derivações*
5	**ROS_DOMAIN_ID fixado no ~/.bashrc (1–100, só seu)**	*setup, passo 5*

Ponto de controle 1 — cole no chat, nesta ordem:

```
git log --oneline -3      +      família do projeto (1–7 / P) e o objeto que a câmera vai procurar
```

Faltou algo? Não para a aula: escreva o sintoma no chat; as tarefas da semana 1 e da Aula 2 continuam valendo e vêm antes da de hoje.

A AULA EM UMA FRASE

Hoje o tópico deixa de carregar texto

Tudo da Aula 2 continua valendo. O que muda é o tipo da mensagem — e, com ele, taxa, latência, perda de quadro e política de entrega.

É a diferença entre uma conversa e um fluxo. Numa conversa, cada mensagem importa e vale a pena repetir a que se perdeu. Num fluxo, a mensagem perdida já nasceu velha: repetir só atrasa a próxima.

Ao fim desta semana (e da próxima, sem aula) você terá rodando o pipeline câmera → segmentação → contagem → serviço sobre o SEU objeto — que é literalmente o esqueleto dos itens 3 e 4 do TP1, entrega em 13/11.

ROTEIRO

Como os 90 minutos se dividem

Minuto	Bloco	O que acontece
0–10	Conferência	ponto de controle 1: tarefa da Aula 2, ambiente, repo, projeto candidato
10–25	O TP1, lido	o que se pede, o que se entrega em 13/11, checklist de verificação, os 7 gates com as datas desta turma
25–50	Imagem no grafo e cor na imagem	sensor_msgs/Image, QoS, cv_bridge, HSV, o vermelho, o pipeline, oclusão — com 2 demos diretas
50–72	Exemplo de ponta a ponta	no pb-live-dacioms: copiar → build → launch → hz → trocar a cor por parâmetro → status → amostrar
72–90	Esta semana e a próxima	guia de estudo, tarefa em duas partes (01/11 e 08/11), desafio 2, socorro, fechamento

02/11 é feriado: a Parte B da tarefa é a aula daquela semana. A Aula 4 (09/11) é a clínica do TP1.

PARTE 1 · 10–25 MIN

O TP1, lido

entrega: sexta 13/11

- o que se pede: as 4 tarefas do enunciado
- o que se entrega: PDF, ZIP, repositório com tag tp1, vídeo ≤ 5 min
- o checklist de verificação por subcompetência (1.1–1.4)
- os 7 gates com as datas desta turma — e o urgente

GATES DO TP1

O TP não é uma entrega: são sete checkpoints — com as datas desta turma

Gate	O que tem de estar verdadeiro	Vence
G1.0	ambiente operante — check-ambiente.sh em 'Tudo pronto!'	dom 01/11
G1.1	projeto declarado: domínio, usuário, DUAS classes, trilha, plano B	dom 01/11
G1.2	imagem no grafo, taxa medida nas duas pontas (ros2 topic hz)	dom 01/11
G1.3	segmentação e contagem sobre o SEU objeto, duas classes	sex 06/11
G1.4	/vision/status com o seu domínio + rqt_graph salvo	dom 08/11
G1.5	relatório com decisões, oclusão discutida, limitações, IA declarada	qua 11/11
G1.6	tag tp1 + PDF, ZIP e links no Moodle	sex 13/11

Um gate ou é verdade ou não é — e se prova com um comando. O bloco pronto para colar no PROJETO.md está na página do TP1. Um commit por gate.

URGENTE

O G1.1 vence domingo 01/11 — é o gate que trava todos os outros

Declare domínio, usuário, classes percebidas, trilha e plano B. Sem isso o G1.3 não tem o que detectar.

Quem não declara o projeto gasta a semana afinando HSV de um círculo vermelho genérico que não vai para lugar nenhum — e depois refaz tudo com o objeto certo, a três dias do prazo.

Duas classes distinguíveis por cor: tampa azul × tampa verde, fruto maduro × verde, capacete × colete. Uma classe só esconde o problema interessante. Sem câmera? Cena sintética com os seus objetos, foto ou vídeo do celular — vale para o TP1 inteiro.

Ponto de controle 2 (chat): 'G1.1 — já sei o domínio e as duas classes: SIM / AINDA NÃO'. Mudar depois é saudável — registrado em docs/decisoes.md.

A RÉGUA É ASSIMÉTRICA DE PROPÓSITO

Simulação × hardware real

Trilha S · simulada

ambiente sob controle

- desafios previsíveis, repetíveis
- sem perda de tempo com driver
- por isso: exige-se MAIS
- mais funções, mais robustez, métrica melhor

Trilha H · híbrida

um meio-termo plausível

- núcleo simulado + UM elemento real
- a webcam já basta
- ruído, luz e latência de verdade

Trilha R · hardware real

bem-vinda — reconhecida no esforço

- o esforço vai para o que a simulação não tem
- resultado pode ser mais modesto
- plano B simulado sempre vivo

Não há trilha recomendada: as três são defensáveis. A certa é a que você sustenta até o TP5 — e declara no PROJETO.md.

PARTE 2 · 25–50 MIN

Imagem no grafo, cor na imagem

em linhas gerais — o detalhe está no guia

- sensor_msgs/Image e a QoS de sensor
- o sintoma cruel: 'existe e não chega' (demo 1)
- cv_bridge e o plano B
- HSV, o vermelho, o pipeline em 5 passos (demo 2), oclusão

ANATOMIA

sensor_msgs/Image não tem mágica nenhuma



É a matriz do OpenCV serializada, mais os metadados de quem vai interpretá-la.

Cabeçalho

quem, quando, em que referencial

- stamp — instante da captura
- frame_id — de qual sensor

```
header.stamp
header.frame_id
```

Geometria

como remontar a matriz

- height, width em pixels
- step = bytes de UMA linha
- is_bigendian

```
step = width * 3
```

Conteúdo

os pixels, e como lê-los

- encoding: bgr8, rgb8, mono8
- data — array contíguo de bytes

```
encoding = 'bgr8'
```

ros2 interface show sensor_msgs/msg/Image — no guia, seção 1. É mais curto do que parece.

POLÍTICA DE ENTREGA

Por que a QoS padrão está errada para câmera

RELIABLE · o padrão do ROS 2

entrega garantida, fila de 10

- retransmite o que se perdeu
- quadro atrasado ainda chega — velho
- a fila acumula e a latência cresce
- certo para comando, errado para sensor

BEST_EFFORT · perfil de sensor

KEEP_LAST, profundidade 1

- quadro perdido é quadro descartado
- sempre o mais recente, nunca uma fila
- latência constante sob carga
- é o que a sua câmera deve usar

Se um quadro se perde, retransmitir é pior do que descartar: quando chegar, já estará velho.

O SINTOMA CRUEL

"O tópico existe, mas não chega nada"

1	**ros2 topic list mostra o tópico — ele está lá**	*tudo parece certo*
2	**ros2 node info mostra os dois nós, publisher e subscriber**	*tudo parece certo*
3	**ros2 topic echo fica em silêncio, para sempre**	*e nenhum erro aparece*
4	**Causa: perfis incompatíveis — BEST_EFFORT × RELIABLE**	*eles nunca se conectam*

O comando que revela os dois perfis lado a lado:

```
ros2 topic info /camera/image_raw --verbose
```

Quando o tópico existe e nada chega, suspeite de QoS antes de qualquer outra coisa. Não há mensagem de erro para isso. (Demo direta 1, agora.)

CONVERSÃO

cv_bridge — e o que fazer quando ele falta

⇄

A ponte entre sensor_msgs/Image e o numpy.ndarray do OpenCV, nos dois sentidos.

O caminho canônico

use este quando existir

- pacote ros-humble-cv-bridge
- imgmsg_to_cv2 e cv2_to_imgmsg
- trata encodings e conversões

```
cv2_to_imgmsg(img, 'bgr8')
```

O plano B do exemplo

módulo ponte.py, sem dependência

- tenta importar; se falhar, monta à mão
- bgr8 = 3 bytes por pixel, B-G-R
- step = largura × 3, data contíguo

```
msg.data = img.tobytes()
```

Um pacote faltando não pode travar a turma. A linha de log do publicador diz qual modo está valendo — e os dois valem. O KeyError: 16 (OpenCV/NumPy de pip) está no guia, seção 7: a cura é fora do venv.

ESPAÇO DE COR

Por que HSV e não BGR

BGR mistura brilho e cor

o problema

- vermelho no sol e vermelho na sombra têm valores completamente diferentes nos TRÊS canais
- "é vermelho" vira uma região torta no espaço, impossível de descrever com limiares
- qualquer mudança de luz quebra a regra

HSV separa as duas coisas

a solução

- H — que cor (matiz)
- S — quão pura (saturação)
- V — quão clara (valor)
- "é vermelho" vira uma faixa estreita em H; a variação de luz vira variação em V, que você tolera com limiar frouxo

Regra prática para começar: aperte o H, afrouxe o S e o V. Depois aperte S e V só o necessário.

A PRIMEIRA ARMADILHA DE VISÃO

No OpenCV, o vermelho mora nas duas pontas

H vai de 0 a 179 — metade do círculo de 360°, para caber em um byte. O vermelho fica perto de 0 E perto de 179.

Consequência direta: segmentar vermelho exige duas máscaras somadas (0–10 e 170–179). Azul, verde e amarelo vivem no meio do intervalo e precisam de uma só.

O sintoma de esquecer isso é traiçoeiro — a detecção funciona "às vezes", porque metade dos tons de vermelho da cena cai na faixa que você escreveu e a outra metade não. Você passa a tarde ajustando S e V achando que o problema é iluminação.

Agora você já conhece a armadilha. Quem não conhece perde, em média, uma tarde inteira com ela.

O PIPELINE

Da imagem à contagem, em cinco passos

1	**cvtColor — BGR para HSV**	*uma linha*
2	**inRange — máscara binária a partir das faixas**	*os seus limiares*
3	**morphologyEx OPEN, depois CLOSE**	*tira ruído, fecha buracos*
4	**findContours em RETR_EXTERNAL**	*só os contornos externos*
5	**filtro de área — descarte o que for pequeno demais**	*o parâmetro que mais rende*

O filtro de área é o parâmetro mais útil e o mais subestimado:

```
python3 teste_offline.py   # a mesma segmentação, sem ROS 2 — e a contagem caindo para 1 em dois quadros
```

OPEN remove pontos isolados; CLOSE fecha buracos. Nessa ordem. O filtro de área é a primeira coisa a ajustar quando a contagem oscila (demo direta 2, agora).

O EXPERIMENTO

A contagem cai de 2 para 1 — e isso é conteúdo

Quando os dois círculos se sobrepõem, o pipeline conta 1. Não é defeito do exemplo: é oclusão, o problema número um de qualquer sistema de percepção real.

Reproduza, registre com print e discuta no relatório do TP1: por que acontece e o que você faria a respeito — área máxima além da mínima, razão de aspecto, memória entre quadros, watershed, ou um detector por aparência no TP2.

Um parágrafo honesto sobre isso vale mais do que um pipeline que finge que o problema não existe — e é um dos tópicos do checklist de verificação do TP1.

Regra geral da disciplina: limitação medida e explicada pontua; limitação escondida não sobrevive à arguição.

AO VIVO · 50–72 MIN

De ponta a ponta, no pb-live-dacioms

- `cp -r …/exemplos/aula03-visao/aula03_visao ros2_ws/src/`
- `colcon build --packages-select aula03_visao --symlink-install`
- `ros2 launch aula03_visao visao.launch.py`
- `ros2 topic hz /vision/contagem` · `hz /camera/image_raw --qos-reliability best_effort`
- `ros2 run rqt_image_view rqt_image_view` → `/vision/segmented` · `rqt_graph`

>_

Fonte sintética por padrão: roda sem webcam, sem usbipd, sem driver. Quem tem ambiente replica junto; o guia (seção 2) repete com as saídas esperadas.

AO VIVO · continuação

Trocar a cor sem tocar no código

>_ *Nada recompilado, nada reiniciado — é isso que a Parte B cobra (zero constante de cor no código). Ponto de controle 3: cole no chat a resposta do service call ou uma linha do hz.*

```
ros2 topic echo /vision/contagem
ros2 param set /segmentador_hsv h_min 100
ros2 param set /segmentador_hsv h_max 130     # caça azul
ros2 service call /vision/status std_srvs/srv/Trigger "{}"
ros2 run aula03_visao amostrar_hsv --ros-args -p imagem:=foto.jpg
```

PARTE 3 · 72–90 MIN

Esta semana e a próxima

02/11 é feriado: não há aula

- guia de estudo: 3 sessões, 3h a 4h
- tarefa em duas partes: A até dom 01/11 · B até dom 08/11
- desafio 2 (opcional): onde errar é grátis
- Aula 4 (09/11): interfaces próprias + clínica do TP1

ATÉ DOM 01/11 (PARTE A) · ATÉ DOM 08/11 (PARTE B)

A tarefa da Aula 3, em duas partes — obrigatória, push na dev

Parte A · até domingo 01/11

fecha G1.0, G1.1 e G1.2

- pacote SEU compilando (renomeado nos 5 lugares)
- exemplo rodando como veio; taxas.md interpretado
- PROJETO.md: G1.1 declarado + bloco de gates
- commits G1.1 / G1.2 na dev

Parte B · até domingo 08/11 (a semana sem aula)

fecha G1.3 e G1.4

- DUAS classes do domínio, limiares medidos
- tudo em YAML — zero constante de cor
- /vision/status com o seu domínio
- oclusão registrada + 4 evidências

Prioridade quando o tempo aperta: ambiente → projeto declarado (G1.1) → Parte A → Parte B → só então o Desafio 2 (métrica em 20 quadros, iluminação quebrada, oclusão, custo em Hz).

SOCORRO RÁPIDO

Os oito erros destas duas semanas

Sintoma	Causa provável e cura
tópico existe, echo/hz mudos	QoS incompatível — ros2 topic info --verbose; --qos-reliability best_effort
KeyError: 16 na conversão	OpenCV/NumPy de pip por baixo do cv_bridge do apt — o exemplo cai no modo manual; cura FORA do venv (guia, seção 7)
rqt_image_view não abre	WSL2: wsl --update · VirtualBox: aceleração 3D ligada ou Guest Additions faltando
webcam não abre	WSL2: usbipd attach · VirtualBox: Extension Pack → Dispositivos → Webcams · enquanto isso: fonte:=sintetico
vermelho quase não detectado	faltou a segunda faixa de matiz, 170–179
contagem oscilando	area_min baixo ou falta OPEN; se os objetos se tocam, é oclusão — registre
editou o YAML, nada mudou	build sem --symlink-install (o launch lê install/share/)

Antes de pedir ajuda no Infnet.Online: comando exato, saída completa, lsb_release -a e ros2 topic info --verbose.

PRÓXIMO ENCONTRO · SEGUNDA 09/11

Aula 4 — interfaces próprias + clínica do TP1

02/11 não tem aula (feriado): é a semana da Parte B · entrega do TP1: sexta 13/11

- Parte A na dev até dom 01/11 · Parte B até dom 08/11
- G1.3 e G1.4 fechados antes da clínica
- uma pergunta específica sobre o SEU projeto
- Ponto de controle 4 agora: 'tarefa A/B + TP1: entendi onde estão'